

Software Engineering – M3-A1

Section A — Concept Application

Question 1 — Development Methodology

For this digital wallet project, I would recommend the Agile methodology. The requirements are changing frequently, and the client wants to review working features every two weeks. Agile supports short development cycles called iterations or sprints, allowing the team to deliver working features regularly, collect client feedback, and adapt the requirements in the next cycle. This makes Agile suitable for a project where requirements are not fixed at the beginning.

If the team chose the wrong methodology, two specific risks would be:

1. Inflexibility and rework: With a rigid Waterfall approach, changes discovered later could require major changes to completed work, increasing time and cost.
2. Late feedback and unsuitable features: If working features are not reviewed regularly, the team may discover late that the product does not meet the client's expectations, causing delays and additional rework.

Question 2 — Git Workflow

The team should use a branch-based Git workflow instead of allowing developers to push unfinished code directly to the main branch. Each teammate should create a separate feature branch from the latest main branch, make and test their changes there, and commit the completed work. The developer should then push the branch and create a Pull Request. Another teammate should review and test the changes before merging them into main. The main branch should be protected so that direct pushes are not allowed.

Important Git commands are:

- `git checkout -b feature-name` — creates and switches to a new feature branch where a developer can work without affecting the main branch.
- `git add .` followed by `git commit -m "message"` — stages the intended changes and records them as a clear versioned commit.
- `git push origin feature-name` — uploads the feature branch to the remote repository so it can be reviewed through a Pull Request.
- `git pull origin main` — gets the latest changes from main so the feature branch can be kept up to date before merging.

Question 3 — HTML5 Form Inputs and Validation

The appropriate HTML input types are:

- Applicant name: text
- Email address: email
- Phone number: tel
- Preferred domain: radio buttons for Web, Data, Mobile and AI
- Resume: file

To make every field compulsory, the required attribute can be added to each required input. For the email field, `type="email"` provides built-in browser validation for the expected email format. For example: `<input type="email" name="email" required>`. The radio group can use required so that one domain must be selected, and the file input can also use required.

These attributes are useful for client-side validation, but they are not sufficient for production-level validation. Browser validation can be bypassed or disabled, and users can send requests directly to the server. Production systems therefore need server-side validation, secure file-upload checks, proper file type/size restrictions, sanitization, and protection against malicious input.

Question 4 — Bootstrap Responsive Grid

Bootstrap's grid system divides a row into 12 columns and uses responsive breakpoint classes to control how much space an element occupies at different screen sizes. By assigning different column sizes for mobile and desktop breakpoints, the same HTML can automatically change its layout without writing custom media-query CSS.

For the article column, I would use:

```
<div class="col-12 col-md-4">...</div>
```

col-12 means the article occupies all 12 columns on small/mobile screens, so one article appears per row. col-md-4 means that from the medium breakpoint and above, the article occupies 4 of the 12 columns. Therefore, three articles fit in one row on desktop ($12 \div 4 = 3$).

Question 5 — Array and Two-Step Rainfall Logic

An array is the correct data structure because the program needs to store 30 readings of the same type of data. An array stores these values in a single named structure and allows them to be accessed using an index such as `rainfall[0]` through `rainfall[29]`. Using 30 separate variables would make the program longer, harder to maintain, and more difficult to process with loops.

The program should use two logical steps:

1. Compute the average: add all 30 rainfall values and divide the total by 30.
2. Find above-average days: traverse the array again and compare each day's rainfall with the calculated average. If a value is greater than the average, print that day's number.

The two operations cannot normally be completed correctly in one simple pass because the comparison value—the final average—is not known until all 30 readings have been processed. A second pass is therefore needed to compare every stored reading against that calculated average.

Question 6 — Pointer-Based String Traversal

When the user presses Enter without typing a name, the input string is empty. A C string is terminated by the null character `'\0'`. If pointer-based code assumes that a character exists and dereferences the pointer without checking the string's termination condition, it can access invalid memory and may crash, depending on how the program is written.

The pointer-based solution must check that the pointer points to a valid character before dereferencing it. During traversal, the normal condition is essentially `*p != '\0'`, with the pointer itself also expected to be valid before use. For example, after reading input, the program can check whether the first character is `'\0'` before processing the string.

With array notation, traversal looks like `str[i]`, while pointer traversal looks like `*p` followed by moving the pointer with `p++`. For empty-input handling, array index notation is often easier to read and debug because checking `str[0] == '\0'` makes the empty-string condition explicit. Pointer traversal is also valid, but it requires careful pointer and termination checks.